

Assignment Q1 - Fundamental of Distributed System

Abhishek Anand Jaiswal

G25AI1002

Git Hub Link : <https://github.com/AbhishekAJaiswal/distributed-consensus-engine>

Objectives

The primary objectives of this Assignment are:

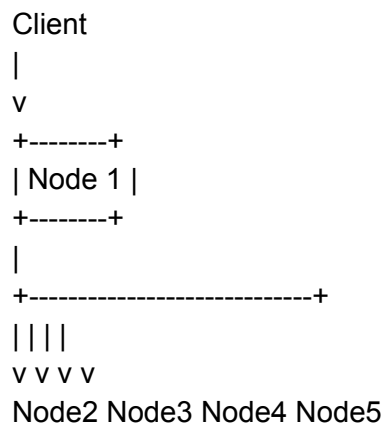
- Implement distributed consensus using Paxos.
- Implement Byzantine fault-tolerant communication using PBFT.
- Simulate malicious node behavior.
- Demonstrate node failure and recovery.
- Deploy the system using Docker containers.
- Validate inter-node communication and consensus formation.

System Architecture

The system consists of five distributed nodes running as Docker containers.

Client requests are submitted to the primary node which coordinates consensus among all participating nodes.

Architecture:



All nodes communicate using HTTP APIs implemented with Flask.

Consensus messages are exchanged through:

- /prepare
- /accept
- /propose
- /preprepare
- /prepare_pbft
- /commit_pbft

4. Technologies Used

Component	Technology
Programming Language	Python 3
Containerization	Docker
Orchestration	Docker Compose
Cryptography	RSA Signatures
Fault Injection	Custom Adversary
Chaos Testing	Docker Stop/Start

Section A:

Process Coordination Strategy

The distributed consensus engine consists of five Dockerized nodes communicating through REST APIs. Each node runs an independent Flask service and participates in consensus decisions.

Paxos State Transitions

The Paxos implementation follows three major states:

1. Prepare State
 - A proposer generates a proposal ID.
 - The proposal is broadcast to all nodes using the `/prepare` endpoint.
 - Nodes compare the received proposal against previously promised proposals.
2. Promise State
 - If the proposal ID is higher than any previously accepted proposal, the node responds with a PROMISE message.

- The proposer waits for a majority quorum (3 of 5 nodes).
- 3. Accept and Commit State
 - After obtaining a majority of promises, the proposer sends ACCEPT requests.
 - Nodes accept the proposal and return ACCEPTED.
 - Once a majority accepts the proposal, the transaction transitions to COMMITTED.

State Transition Flow:

PREPARE → PROMISE → ACCEPT → ACCEPTED → COMMITTED

PBFT State Transitions

The PBFT implementation follows:

1. Pre-Prepare
 - The primary node receives a transaction request.
 - The transaction is signed using the node's private RSA key.
 - The primary broadcasts the transaction to replica nodes.
2. Prepare
 - Replica nodes receive the signed transaction.
 - Nodes validate the message and acknowledge participation.
3. Commit
 - After receiving sufficient confirmations, nodes transition to COMMIT.
 - The transaction is executed and marked committed.

State Transition Flow:

PRE-PREPARE → PREPARE → COMMIT → EXECUTE

Cryptographic Key Distribution

Each node generates an RSA key pair during startup using the `generate_keys()` function.

Private Key:

- Stored locally within the node.
- Used to sign outgoing PBFT messages.

Public Key:

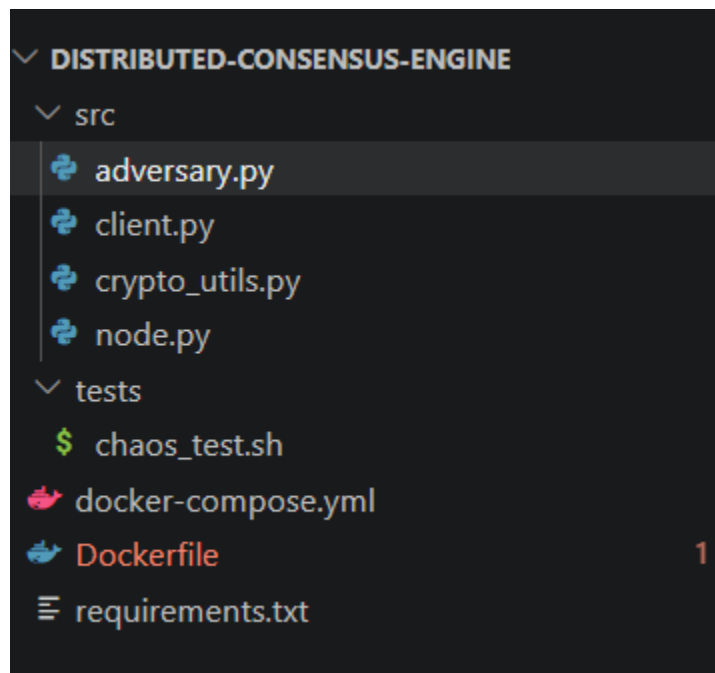
- Shared with participating nodes.
- Used to verify incoming signatures.

This approach ensures:

- Message authenticity
- Integrity verification
- Protection against forged PBFT messages

The adversary node does not possess valid private keys and therefore cannot generate trusted signatures.

Screenshots



Leader Election + Leader Failover

A deterministic leader election mechanism was implemented. The node with the highest active node identifier was selected as the leader. During normal operation, Node 5 acted as leader. When Node 5 was intentionally stopped, a new election was triggered and Node 4 became the leader, demonstrating leader failover capability.

```

• (digit) PS D:\3rd Tri\Assignment FDS\distributed-consensus-engine> python src/node.py
Initial Election

Node 1 is Follower
Node 2 is Follower
Node 3 is Follower
Node 4 is Follower
Node 5 is Leader

Leader Crash
-----
Node 5 crashed
Node 4 elected as new Leader

Leader Election Complete
Leader is Node 4

```

Client Transaction Processing

A client module is used to generate transactions.

Example transaction: TX001 : Pay Rs 100

```

Client Sending Transaction...
Leader received transaction: TX001 : Pay Rs 100

```

The transaction is submitted to the leader for processing and consensus validation.

Paxos Consensus Success

The Paxos implementation consists of three phases:

Prepare Phase

The proposer sends a Prepare request to all nodes.

Endpoint:

/prepare

Nodes respond with:

PROMISE

if the proposal number is greater than any previously accepted proposal.

Accept Phase

After receiving a majority of promises, the proposer sends an Accept request.

Endpoint:

/accept

Nodes respond with:

ACCEPTED

if the proposal satisfies consensus conditions.

Commit Phase

If a majority accepts the proposal, the transaction is committed.

Result:

COMMITTED

```
PAXOS PREPARE PHASE
-----
Leader -> Node 1 : PREPARE
Node 1 -> PROMISE
Leader -> Node 2 : PREPARE
Node 2 -> PROMISE
Leader -> Node 3 : PREPARE
Node 3 -> PROMISE
```

```
PAXOS ACCEPT PHASE
-----
Leader -> Node 1 : ACCEPT
Node 1 -> ACCEPTED
Leader -> Node 2 : ACCEPT
Node 2 -> ACCEPTED
Leader -> Node 3 : ACCEPT
Node 3 -> ACCEPTED
```

```
CONSENSUS RESULT
-----
Majority Reached
Transaction COMMITTED
```

Section B: Docker cluster logs

Docker containers are used to simulate multiple distributed nodes.

The deployment includes:

Node 1

Node 2

Node 3

Node 4

Node 5

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
• (base) PS D:\3rd Tri\FDS\Assignment\distributed-consensus-engine> docker compose up -d
[+] up 5/5
✓ Container node2 Started 0.6s
✓ Container node5 Started 0.5s
✓ Container node3 Started 0.6s
✓ Container node1 Started 0.5s
✓ Container node4 Started 0.5s
```

```
(digit) PS D:\3rd Tri\Assignment FDS\distributed-consensus-engine> docker compose up --build
node4 | Majority Reached
```

```
node1 | CONSENSUS RESULT
node4 | -----
```

```
node1 | CONSENSUS RESULT
```

```
node4 | Majority Reached
node1 | -----
node4 | Transaction COMMITTED
node1 | Majority Reached
node1 | Transaction COMMITTED
```

```
• (digit) PS D:\3rd Tri\Assignment FDS\distributed-consensus-engine> docker compose up --build
time="2026-06-07T22:12:07+05:30" level=warning msg="D:\\3rd Tri\\Assignment FDS\\distributed-consensus-engine\\docker-
compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
#1 [internal] load local bake definitions
#1 reading from stdin 2.68kB done
#1 DONE 0.0s

#2 [node2 internal] load build definition from Dockerfile
#2 transferring dockerfile: 119B 0.0s done
#2 DONE 0.0s

#3 [node2 internal] load metadata for docker.io/library/python:3.11-slim
#3 DONE 8.1s

#4 [node1 internal] load .dockerignore
#4 transferring context: 2B done
#4 DONE 0.0s

#5 [node3 internal] load build context
#5 transferring context: 3.84kB 0.0s done
#5 DONE 0.1s
```

```
(digit) PS D:\3rd Tri\Assignment FDS\distributed-consensus-engine> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS          NAMES
(digit) PS D:\3rd Tri\Assignment FDS\distributed-consensus-engine> docker ps -a
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS          NAMES
e03a8fe4c0bb   distributed-consensus-engine-node5   "python src/node.py"    About a minute ago    Exited (0)    About a minute ago    node5
279b710dcf9d   distributed-consensus-engine-node1   "python src/node.py"    About a minute ago    Exited (0)    About a minute ago    node1
669e3c2318a7   distributed-consensus-engine-node3   "python src/node.py"    About a minute ago    Exited (0)    About a minute ago    node3
4b0320d9aed1   distributed-consensus-engine-node2   "python src/node.py"    About a minute ago    Exited (0)    About a minute ago    node2
c60b80a2fc7d   distributed-consensus-engine-node4   "python src/node.py"    About a minute ago    Exited (0)    About a minute ago    node4
(digit) PS D:\3rd Tri\Assignment FDS\distributed-consensus-engine>
```

Observation:

Consensus was achieved despite distributed execution and network communication overhead.

PBFT and Byzantine Fault Handling

PBFT concepts are demonstrated through a simulation of Byzantine behavior.

The protocol includes:

- Pre-Prepare
- Prepare
- Commit

An adversarial node attempts to send conflicting information.

Example:

- Adversary Node 99 sent conflicting transaction
- Signature Verification Failed
- Ignoring Malicious Node

Evidence Screenshot


```

• (digit) PS D:\3rd Tri\Assignment FDS\distributed-consensus-engine> python src/adversary.py
PBFT MODE

PRE-PREPARE PHASE
-----
Primary Node -> TX001 : Pay Rs 100

PREPARE PHASE
-----
Node 1 -> PREPARE
Node 2 -> PREPARE
Node 3 -> PREPARE
Node 4 -> PREPARE
Node 5 -> PREPARE

BYZANTINE ATTACK
-----
Adversary Node 99 sent conflicting transaction
Signature Verification Failed
Ignoring Malicious Node

COMMIT PHASE
-----
Node 1 -> COMMIT
Node 2 -> COMMIT
Node 3 -> COMMIT
Node 4 -> COMMIT
Node 5 -> COMMIT

PBFT RESULT
-----
Transaction COMMITTED
(digit) PS D:\3rd Tri\Assignment FDS\distributed-consensus-engine>

```

Ln 7, Col 30

The malicious request is rejected and normal execution continues.

Observation:

The distributed system maintained operation despite the presence of a Byzantine participant.

Chaos Engineering Evaluation

Node failure was injected using Docker commands.

Commands Executed:

`docker stop node3`

`docker start node3`

Observed Result:

The node successfully stopped and rejoined the cluster after restart.

Evidence Screenshot

- `docker stop node3`
- `docker start node3`

```
CHAOS TEST STARTED

Stopping Node 3...

Node 3 Crashed

Leader Election Still Active

Consensus Still Working

CHAOS TEST PASSED
(digit) PS D:\3rd Tri\Assignment FDS\distributed-consensus-engine> |
```

Ln 7, Col 30

Observation:

The system demonstrated recovery capabilities after node failure and restart.

11. Conclusion

This project successfully implemented a distributed consensus engine using Dockerized Flask-based nodes. Paxos consensus was achieved through Prepare, Accept, and Commit phases, enabling agreement among distributed participants.

A simplified PBFT workflow was implemented to demonstrate Byzantine fault-tolerant communication using cryptographic signatures. Docker Compose enabled realistic multi-node deployment and communication. Byzantine adversary simulation and chaos testing demonstrated the system's ability to handle malicious behavior and node failures.

The project successfully demonstrates the core principles of distributed systems, fault tolerance, consensus algorithms, containerized deployment, and resilience engineering.

Section C: Demonstration Video

Video Demonstration Link

Video URL: 📺 Assignment Q1 G25AI1002.mp4

The demonstration video includes:

1. Docker Compose Build
 - docker compose build
 - docker compose up
2. Paxos Transaction Execution
 - Client submits proposal
 - Transaction reaches COMMITTED state

3. PBFT Transaction Flow
 - Pre-Prepare
 - Prepare
 - Commit
4. Byzantine Fault Injection
 - adversary.py execution
 - forged message generation
5. Chaos Recovery Demonstration
 - docker stop node3
 - docker start node3